



Sistemas Operativos

Universidade do Minho

2025/2026

Projeto de Grupo

“Sistema Cliente-Servidor Simples para a Execução de Programas”

Turno TP1

Grupo 24

104359 — Gabriela Sousa e Santos — LEGSI

111787 — Matilde Pires Cunha — LEGSI

110943 — Mariana Silva Gomes — LEGSI

Índice

1. Introdução.....	3
1.1. Contextualização.....	3
1.2. Objetivos do Projeto	3
2. Metodologia e Abordagem Técnica.....	4
2.1. Estrutura do Código e Modularidade.....	4
2.2. Restrições de Implementação	4
3. Funcionamento e Arquitetura do Sistema.....	6
3.1. Visão Geral	6
4. Comunicação entre Processos	8
4.1. Named Pipes e Persistência.....	8
4.2. Protocolo de Comunicação e Parsing Manual	8
5. Motor de Execução	10
5.1. Criação de Processos	10
5.2. Execução de Comandos.....	10
5.3. Sincronização	11
6. Sistema de Logs e Persistência.....	12
6.1. Formato e Estrutura dos Registos.....	12
6.2. Obtenção do Estado de Execução.....	12
6.3. Persistência e Acumulação de Dados	13
7. Testes e Validação.....	14
8. Ferramentas de Apoio e Fontes de Informação Técnica.....	16
9. Conclusão.....	17

1. Introdução

1.1. Contextualização

Este relatório descreve o desenvolvimento do projeto da Unidade Curricular de Sistemas Operativos, inserida no 2º ano da Licenciatura em Engenharia e Gestão de Sistemas de Informação da Universidade do Minho.

Este trabalho consistiu no desenvolvimento de um sistema cliente-servidor simples em linguagem C para a execução de programas não interativos em ambiente Linux.

1.2. Objetivos do Projeto

O presente projeto tem como foco central o desenvolvimento e a implementação de um sistema distribuído robusto, projetado especificamente para a execução remota de tarefas. Para atingir este objetivo, definimos um conjunto de metas técnicas, estritamente interligadas, que exigem um domínio aprofundado dos fundamentos dos sistemas operativos. Estas incluem:

- **Comunicação entre Processos (IPC):** Utilização de Named Pipes (FIFOs) para o intercâmbio de dados entre processos independentes.
- **Gestão de Processos:** Controlo total do ciclo de vida de processos (criação, execução e sincronização) através das primitivas `fork()`, `execvp()` e `waitpid()`.
- **Gestão de Ficheiros e Persistência:** Implementação de um sistema de logging robusto recorrendo a descritores de ficheiro e manipulação de fluxos de dados.
- **Programação de Baixo Nível:** Cumprir a restrição estrita de utilizar apenas chamadas ao sistema estudadas (como `read()`, `write()`, `open()`, `close()`), evitando deliberadamente funções de alto nível da biblioteca padrão (como `fprintf()` ou `fopen()`) e a utilização de threads.

2. Metodologia e Abordagem Técnica

2.1. Estrutura do Código e Modularidade

A solução foi consolidada num único ficheiro fonte. Através da função main, o programa atua como um "despachante", decidindo o papel a assumir (Cliente ou Servidor) com base nos argumentos passados no terminal. O código está organizado em funções especializadas para garantir a legibilidade:

- **Divisão Funcional:** O programa evita lógica monolítica, distribuindo responsabilidades por blocos modulares (ex: executarComandos).
- **Cliente (Emissor):** Especializado na interface com o utilizador, recolha de argumentos via argv e encapsulamento de dados para o canal de comunicação.
- **Servidor (Processador):** Atua como o núcleo do sistema, contendo a lógica de gestão do FIFO e o motor de execução.

Internamente, o código foi organizado de forma a separar a lógica de parsing (preparação e limpeza de comandos) da lógica de execução (gestão de processos). Esta modularidade permitiu testar a robustez do tratamento de strings antes de integrar a criação de processos, garantindo que apenas comandos devidamente formatados são enviados para execução.

2.2. Restrições de Implementação

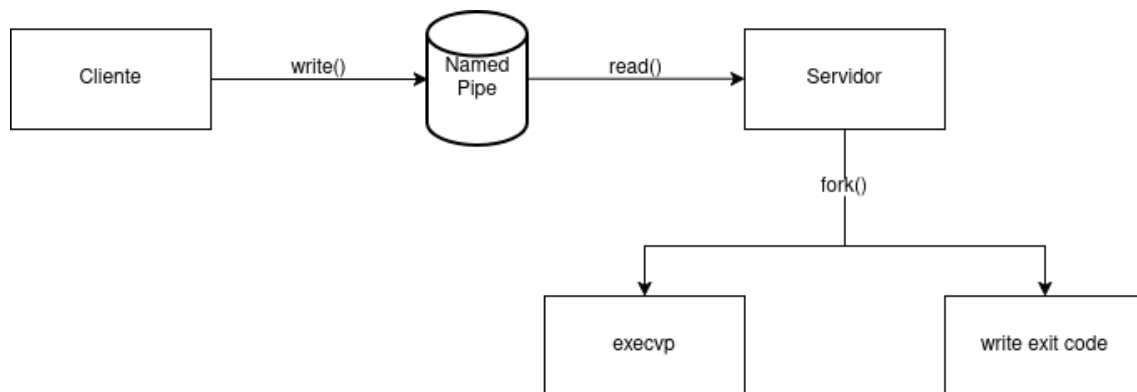
O cumprimento rigoroso das restrições pedagógicas da unidade curricular foi o motor principal de toda a implementação. Optamos por uma estratégia de proximidade ao Kernel, privilegiando a interação direta com o sistema operativo em detrimento das facilidades oferecidas por bibliotecas de alto nível.

- **Prioridade às Chamadas ao Sistema:** Substituiu-se o uso de funções com buffering (como fprintf) pela system call write (direcionada ao STDERR_FILENO), garantindo um I/O direto e sem intermediação.
- **Abstenção de Funções de Alto Nível:** Evitaram-se funções como system() ou popen(). O controlo da execução foi implementado manualmente através da combinação de fork() e execvp().

- **Gestão Manual de Dados e Concorrência:** O sistema não utiliza threads. Devido à interdição do `sprintf`, a conversão do código de saída (inteiro) para texto (ASCII) no ficheiro de log foi realizada através de um algoritmo manual de divisões sucessivas e inversão de caracteres.
- **Persistência de Baixo Nível:** A manipulação de ficheiros de log foi feita via `open()`, utilizando as flags `O_WRONLY`, `O_CREAT` e `O_APPEND`. Esta configuração assegura que os registos são anexados atómicamente ao final do ficheiro, preservando a integridade do histórico.

3. Funcionamento e Arquitetura do Sistema

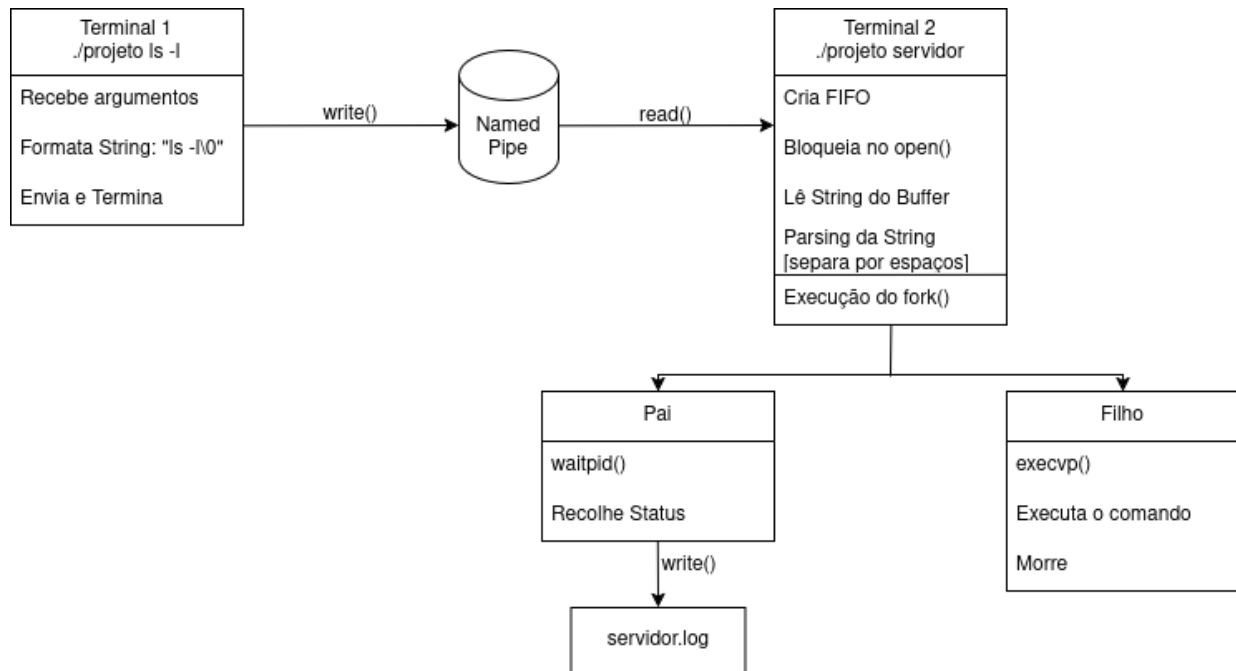
3.1. Visão Geral



A arquitetura do sistema baseia-se no modelo Produtor-Consumidor. O Cliente atua como o Produtor, sendo responsável por captar os argumentos da linha de comandos, formatá-los numa mensagem única e escrevê-los no canal de comunicação. O Servidor atua como Consumidor, processando os pedidos de execução de forma isolada.

Uma restrição arquitetural importante, respeitada nesta implementação, é a de que apenas um cliente interage com o sistema de cada vez, garantindo a integridade da comunicação no canal FIFO e a ordem dos registos no ficheiro de log.

3.2. Fluxo de Execução



O fluxo de execução inicia-se no Terminal 2 (Servidor), que é responsável por estabelecer a infraestrutura de comunicação através da system call `mkfifo`. De seguida, o servidor entra num estado de bloqueio na chamada `open(FIFO_NAME, O_RDONLY)` a aguardar a abertura do canal por um cliente.

Quando o Terminal 1 (Cliente) é executado, este envia a string formatada via `write` e termina. O servidor desbloqueia, lê a mensagem e inicia o processamento sequencial:

1. O processo pai invoca `fork()` para criar um processo filho.
2. O processo filho substitui a sua imagem de memória pelo comando solicitado através de `execvp()`.
3. O processo pai aguarda a conclusão do filho (`waitpid`), recolhe o exit status e regista o resultado no ficheiro `servidor.log` através da system call `write`.

4. Comunicação entre Processos

4.1. Named Pipes e Persistência

A escolha de Named Pipes (ou FIFOs) como mecanismo de IPC foi determinada pela necessidade de estabelecer um canal entre processos independentes, sem relação de parentesco. Ao contrário dos pipes anónimos, os named pipes possuem um nome associado no sistema de ficheiros, permitindo que o Cliente e o Servidor se encontrem através de um caminho comum, neste caso, o ficheiro especial definido como `pipe_so`

A implementação segue o ciclo de vida padrão destas primitivas:

- **Criação:** O servidor utiliza a system call `mkfifo()` com permissões `0666`, criando um objeto que, embora visível no sistema de ficheiros, é gerido como um buffer transiente pelo Kernel.
- **Sincronização na Abertura:** O Servidor abre o pipe em modo `O_RDONLY`. Por norma, esta chamada bloqueia o servidor até que um processo "Produtor" abra o mesmo pipe. O Cliente abre o canal em `O_WRONLY` e coloca a "encomenda" via `write()`.
- **Limpeza de Recursos:** Ambos os processos encerram os seus descritores com `close()`. No término da execução, o servidor invoca `unlink()`. Esta etapa é crítica, pois, ao contrário da memória volátil, o objeto FIFO persiste no disco, podendo causar erros em execuções futuras se não for devidamente removido.

4.2. Protocolo de Comunicação e Parsing Manual

Para garantir a robustez na troca de dados, estabeleceu-se um protocolo de mensagens específico que cumpre o requisito de distinguir claramente os programas e os seus respetivos argumentos:

1. **"Empacotamento" no Cliente:** O programa Cliente agrupa todos os argumentos de `argv` numa string única (encomenda), utilizando espaços como delimitadores. Esta estratégia permite o envio de múltiplos comandos (separados por `;`) numa única operação de `write()`, incluindo o envio explícito do terminador nulo (`\0`) para sinalizar o fim da mensagem no buffer do Kernel. Esta abordagem reduz o overhead de chamadas ao sistema e otimiza a comunicação IPC.

2. **Parsing de Dois Níveis no Servidor:** Para processar a mensagem, o Servidor executa uma análise manual em duas etapas:

- **Nível de Mensagem:** O servidor percorre o buffer e identifica o caractere ; para isolar cada comando individual na variável `comando_atual`.
- **Nível de Comando:** Através da função `preparar_e_executar`, o servidor segmenta o nome do programa dos seus argumentos.

Em vez de alocar novos buffers (o que seria ineficiente), o código realiza uma modificação in-place, percorre a string original, em que salta espaços e substitui-os pelo terminador nulo (`\0`). Esta técnica "corta" a string original em múltiplos sub-blocos de memória, criando um array de apontadores (`char *args[]`) onde o último elemento é explicitamente `NULL`. Este formato é o requisito estrito para que a chamada `execvp()` consiga identificar o fim da lista de argumentos sem aceder a memória inválida.

5. Motor de Execução

5.1. Criação de Processos

Para executar comandos externos sem interromper o funcionamento do servidor, utiliza-se a chamada ao sistema `fork()`. Esta primitiva cria uma cópia exata do processo servidor (o processo pai), gerando um novo processo independente (o processo filho) que, embora herde os descritores de ficheiros e o contexto, possui o seu próprio espaço de endereçamento.

No código desenvolvido, o valor de retorno do `fork()` é verificado para determinar o fluxo de execução:

- Se devolver 0, o código identifica que está no contexto do processo filho, onde será carregado o novo programa.
- Se o retorno for positivo (o PID do filho), o código identifica que está no processo pai (servidor), mantendo-se em execução para gerir o ciclo de vida do pedido.

Esta separação é crucial para garantir que o servidor permanece ativo e capaz de continuar a ler do pipe após a execução de um comando.

5.2. Execução de Comandos

No contexto do processo filho, a escolha específica do `execvp()` da família `exec()` deve-se a dois fatores técnicos determinantes para o projeto:

1. A função aceita um array de strings (`char *args[]`) como argumento. Isto integra-se perfeitamente com a lógica de parsing implementada, que converte a string recebida do cliente num vetor de palavras (ex: `{"ls", "-l", NULL}`).
2. A função procura automaticamente o executável nas diretorias listadas na variável de ambiente `PATH`. Isto permite ao cliente enviar comandos simples como `ls` ou `ps`, sem a necessidade de o utilizador conhecer ou escrever o caminho absoluto, aumentando a usabilidade do sistema.

Ao ser invocada com sucesso, esta chamada substitui integralmente a imagem de memória do processo filho pelo novo programa, que inicia a sua execução.

5.3. Sincronização

A coordenação entre o servidor e os programas em execução é garantida pela system call `waitpid()`. No código implementado, o processo pai invoca esta chamada imediatamente após o `fork()`, passando o PID do filho e bloqueando a sua própria execução até que esse processo específico termine.

Embora o enunciado mencione execução concorrente, a colocação estratégica do `waitpid()` logo após a criação de cada processo filho assegura um processamento sequencial e ordenado (um comando de cada vez).

6. Sistema de Logs e Persistência

6.1. Formato e Estrutura dos Registos

O sistema de logging foi desenhado para fornecer um rasto detalhado e legível de todas as atividades processadas pelo servidor. Cada entrada no ficheiro de log segue um formato estritamente consistente: <programa> <argumentos>; exit status: <valor>

A construção desta linha é realizada de forma incremental na memória (RAM) antes de qualquer operação de I/O:

1. **Segmento de Comando:** O servidor percorre o array de argumentos (`args[]`), concatenando cada elemento num *buffer* intermédio e separando-os por espaços para reconstruir a invocação original.
2. **Metadados:** É anexada a string fixa ; exit status: para delimitar o comando do seu desfecho.
3. **Finalização:** O código de terminação (convertido manualmente) é inserido, terminando com um carácter de nova linha (`\n`) para garantir a legibilidade do histórico.

6.2. Obtenção do Estado de Execução

A fiabilidade do log depende da captura precisa do desfecho do processo filho. O servidor utiliza a infraestrutura de sincronização do *Kernel* para este efeito:

- **`waitpid(pid, &status, 0)`:** Esta chamada bloqueia o servidor até que o processo filho termine, preenchendo a variável `status` (um mapa de bits) com metadados sobre a terminação.
- **Macro `WIFEXITED(status)`:** Utilizada para confirmar se o processo terminou de forma voluntária (ex: via `return` ou `exit`).
- **Macro `WEXITSTATUS(status)`:** Extrai o código de saída real (0-255). É este valor que permite ao utilizador saber se o comando foi bem-sucedido (0) ou se ocorreu um erro específico.

6.3. Persistência e Acumulação de Dados

Para garantir que o histórico sobrevive ao encerramento do servidor, utilizou-se a system call `open()` com uma configuração de flags que assegura a persistência robusta:

- **O_CREAT:** Garante a criação automática do ficheiro caso este seja inexistente.
- **O_APPEND:** Esta é a flag crítica; ela instrui o Kernel a posicionar o apontador de escrita sempre no final do ficheiro antes de cada operação, garantindo que novos registos são anexados atomicamente sem sobrescrever o histórico.
- **Permissões 0644:** Definidas para permitir leitura/escrita ao dono e apenas leitura a terceiros, respeitando normas de segurança.

A escrita final utiliza a chamada `write()`, transferindo os dados diretamente para o sistema de ficheiros. Esta abordagem evita os mecanismos de buffering da biblioteca padrão, reforçando a integridade dos dados em caso de falha do sistema.

Destaque Técnico: Conversão Manual de Dados (int to string)

Uma das maiores complexidades deste módulo foi a **conversão manual de dados**. Dada a interdição do uso de `sprintf()`, o sistema implementa um algoritmo de conversão ASCII de baixo nível:

1. **Extração:** Através de divisões sucessivas por 10 e operações de módulo (%), cada dígito do exit status é isolado.
2. **Mapeamento:** Cada valor numérico é convertido no seu correspondente carácter ASCII somando o deslocamento '0'.
3. **Inversão:** Como os dígitos são extraídos do menos significativo para o mais significativo, o código executa uma rotina de inversão de string para garantir a ordem de leitura correta.

7. Testes e Validação

A validação do sistema seguiu uma abordagem incremental, garantindo o funcionamento dos componentes básicos antes da integração final.

Criou-se manualmente o pipe e iniciou-se a leitura com o comando cat.

```
~/S0/projeto_S0$ mkfifo pipe_so
~/S0/projeto_S0$ cat pipe_so
```

O terminal ficou bloqueado, a aguardar dados, tal como era esperado.

Num segundo terminal, enviou-se uma mensagem de teste, o que resultou no desbloqueio imediato do primeiro terminal e na apresentação da mensagem. Este teste preliminar confirmou que a persistência do FIFO no sistema de ficheiros e a sincronização de leitura/escrita estavam a funcionar corretamente

Num segundo terminal, enviou-se uma mensagem de teste (echo "ls -l" > pipe_so), o que resultou no desbloqueio imediato do primeiro terminal e na apresentação da mensagem. Este teste preliminar confirmou que a persistência do FIFO no sistema de ficheiros e a sincronização de leitura/escrita estavam a funcionar corretamente.

```
~/S0/projeto_S0$ echo "ls -l" > pipe_so
```

Após a validação manual do canal, avançou-se para os testes com os programas desenvolvidos (./projeto). O servidor foi iniciado no Terminal 1, que fica bloqueado a espera, enquanto no Terminal 2 se executou o cliente com argumentos variados.

```
[Servidor] Pronto para iniciar o processamento.
Received: pwd
Comando isolado: 'pwd'
/home/gabi/S0/projeto_S0
Received: ps
Comando isolado: 'ps'
  PID TTY          TIME CMD
  44404 pts/1    00:00:00 bash
  44466 pts/1    00:00:00 projeto
  44546 pts/1    00:00:00 ps
Received: ls -lisa
Comando isolado: 'ls -lisa'
total 48
34866786  4 drwxrwxr-x 2 gabi gabi  4096 jan 11 12:16 .
34866958  4 drwxrwxr-x 6 gabi gabi  4096 dez 28 16:23 ..
34865650  0 prw-rw-r-- 1 gabi gabi    0 jan 11 12:16 pipe_so
34865272 24 -rwxrwxr-x 1 gabi gabi 20824 jan 11 11:39 projeto
34865926 12 -rw-rw-r-- 1 gabi gabi 12265 jan 11 11:40 projeto.c
34866574  4 -rw-r--r-- 1 gabi gabi   217 jan 11 12:16 servidor.log
```

```

3 gabi@Gabi-YOGA:~/S0/projeto_S0$ ./projeto pwd
4 [Cliente] comando enviado: pwd
5 gabi@Gabi-YOGA:~/S0/projeto_S0$ ./projeto ps
6 [Cliente] comando enviado: ps
7 gabi@Gabi-YOGA:~/S0/projeto_S0$ ./projeto ls -lisa
8 [Cliente] comando enviado: ls -lisa

```

Por fim, para validar o requisito de execução concorrente, realizou-se um teste de stress enviando múltiplos comandos simultâneos com tempos de execução díspares:

```

1 gabi@Gabi-YOGA:~/S0/projeto_S0$ ./projeto sleep 5 ";" ls -l
2 [Cliente] comando enviado: sleep 5 ; ls -l
3 gabi@Gabi-YOGA:~/S0/projeto_S0$ ./projeto "sleep 5; ls -l"
4 [Cliente] comando enviado: sleep 5; ls -l

```

```

Received: sleep 5 ; ls -l
Comando isolado: 'sleep 5 '
Comando isolado: ' ls -l'
total 40
prw-rw-r-- 1 gabi gabi      0 jan 11 12:27 pipe_so
-rwxrwxr-x 1 gabi gabi 20824 jan 11 11:39 projeto
-rw-rw-r-- 1 gabi gabi 12265 jan 11 11:40 projeto.c
-rw-r--r-- 1 gabi gabi   296 jan 11 12:27 servidor.log

```

O servidor realizou o parsing corretamente, identificando e isolando os dois comandos. Observou-se que o processamento ocorreu de forma sequencial: o servidor executou o comando sleep 5, aguardou a sua terminação completa (bloqueando o processo pai), registrou o respetivo exit status no log, e só depois iniciou a execução do comando seguinte (ls -l).

Embora o objetivo inicial contemplasse a execução concorrente, optou-se por uma implementação iterativa segura onde o waitpid é invocado imediatamente após o fork. Esta decisão de projeto priorizou a integridade dos dados, garantindo a escrita atômica e ordenada no ficheiro servidor.log e eliminando o risco da ocorrência de race conditions ou de processos zombie, de modo a assegurar um sistema mais robusto e previsível.

8. Ferramentas de Apoio e Fontes de Informação Técnica

A utilização de ferramentas de IA generativa (especificamente Gemini e NotebookLM) foi adotada estritamente como um mecanismo de consulta técnica e depuração (debugging), funcionando como um complemento à documentação oficial. Em conformidade com as instruções pedagógicas, a IA não foi utilizada para a geração integral da solução, mas sim para esclarecer a semântica de chamadas ao sistema e validar lógicas de baixo nível.

As principais questões colocadas à IA incluíram:

- "Como realizar a conversão manual de um int para ASCII sem utilizar a função `sprintf()`?"
- "Como garantir que a escrita no ficheiro de log seja atômica e não sobrescreva registos anteriores utilizando `open()`?"
- "Como implementar um parsing manual de strings em C substituindo espaços por terminadores nulos in-place?"

A ferramenta apoiou o grupo na compreensão das macros de estado do **`waitpid()`** (como **`WIFEXITED`** e **`WEXITSTATUS`**) e na estruturação da lógica de sincronização do pipe, garantindo que o servidor bloqueasse corretamente na leitura até à chegada de um cliente.

O desenvolvimento baseou-se primordialmente nos materiais curriculares e em documentação padrão de sistemas Unix:

1. Materiais da Unidade Curricular: O código seguiu os exemplos e as boas práticas detalhadas nos guiões das atividades práticas e nos handouts teóricos, especificamente no que toca à gestão de processos com **`fork()`** e comunicação via **Named Pipes**.

3. Documentação GNU C Library: Consultou-se a documentação oficial para compreender a manipulação de descritores de ficheiros e a semântica de bloqueio em pipes.

4. Adaptação de Código: Não foi integrado código de bibliotecas externas adicionais. Toda a lógica de processamento de mensagens e gestão de ficheiros foi implementada utilizando as bibliotecas padrão da linguagem C (`<unistd.h>`, `<sys/wait.h>`), privilegiando sempre a interface de chamadas ao sistema em detrimento de funções de alto nível

9. Conclusão

O projeto consolidou conceitos de IPC e gestão de processos no Linux através de um exercício de proximidade ao Kernel. Ao abdicar de abstrações de alto nível, superaram-se desafios críticos como o parsing manual de comandos (in-place) e a conversão manual de dados para log, reforçando o domínio sobre manipulação de apontadores e representação de dados em memória.

A arquitetura baseada em Named Pipes e o ciclo fork-exec-wait foram fundamentais para compreender a sincronização do sistema e o isolamento de espaços de endereçamento.

Em suma, os objetivos de aprendizagem foram atingidos, resultando numa ferramenta robusta que assegura a integridade da comunicação e a persistência ordenada dos registos em disco.